Table of Contents

1. Executive Summary
2. System Architecture Overview
3. UML Diagrams
4. Data Models
5. System Workflows
6. API Design

1. Executive Summary

This technical blueprint defines the architecture and design for an AI-Powered Exam System that generates unique essay exam questions in real-time using Large Language Models (LLMs) and automatically grades student responses. The system addresses challenges in modern education including question bank memorization, cheating prevalence, and the inability to adapt assessments to diverse student perspectives and grading styles.

Key Objectives:

- Generate unique essay questions with grading rubrics on demand
- Automatically grade student responses with detailed feedback
- Prevent question memorization and cheating
- Provide extensible architecture for future enhancements

Architecture Highlights: The system employs a modular design, leveraging a gateway pattern, audit-enhanced event sourcing with successful grading only-to-ledger optimization, and distributed caching strategies. Within grading workflows, sequential processing ensures consistent rubric application across the generated exam corpus, providing criterion-by-criterion feedback, strengths, weaknesses, and improvement suggestions.

2. System Architecture Overview

2.1 High-Level Architecture

The system follows a layered architecture:

2.2 Core Components

2.2.1 Frontend Layer

- **Web Portal:** Teacher/Admin UI(React/VUE, with SSR)/ templates
- **Mobile App:** Student interface
- **Pages:** Home/dash, create, exam/list, exam/id, results/id
- **Communication:** Rest+websocket(real-time updates)

2.2.2 Backend Layer

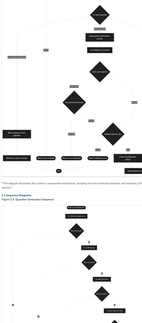
- **Runtime:** Node.js/Python 3.11+ / Go
- **Language:** Python 3.11+
- **Framework:** FastAPI/Express/Gin
- **API:** RESTful request handling and routing , Event-driven management for exam delivery , LLM prompt construction and API integration , Response parsing and validation , Rubric match management

2.2.3 LLM Integration

- **Primary:** OpenAI GPT-4
- **Backup:** Anthropic Claude/ Azure OpenAI
- **Strategy:** Retry with exponential backoff
- **Fallback:** Switch to backup LLM on failure

2.3 Data Flow

Figure 2.3: Data Flow Diagram



This diagram shows all classes, their attributes, methods, and relationships. It includes data models (ExamResponse, QuestionResponse, GradingResult), and enum types (QuestionStatus, QuestionType, ExamStatus, ExamResponseStatus).

Key Relationships:

- ExamResponse contains multiple QuestionResponse, GradingResult
- QuestionResponse has a GradingResult and belongs to Question
- GradingResult uses Rubric and Question
- Exam contains multiple Question

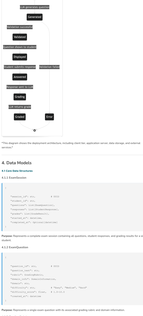
2.4 Grading Workflow

Figure 2.4: Grading Workflow Sequence

This diagram illustrates the system's complete workflow, showing how the frontend, backend, and external services interact.

2.5 Sequence Diagram

Figure 2.5: System Sequence Diagram



This sequence diagram shows the complete interactions between all system components, including exam creation, student exam taking, and results viewing.

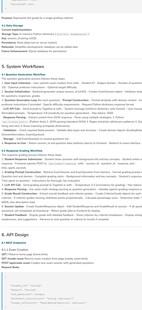
2.6 Activity Diagram

Figure 2.6: Question Generation Activity Flow

This activity diagram shows the decision points and flow control for question generation, including error handling and validation steps.

2.7 Grading Workflow Activity Flow

Figure 2.7: Grading Workflow Activity Flow



This activity diagram shows the complete grading workflow, including rubric matching, error handling, and statistics calculation.

2.8 Component Diagram

Figure 2.8: System Component Diagram

This diagram shows all system components from both professor and student perspectives, including each criteria taking exams, and viewing results.

2.9 Deployment Diagram

Figure 2.9: System Deployment

This diagram shows the deployment architecture, including client tier, application server, data storage, and external services.

4. Data Models

4.1 Core Data Structures

4.1.1 ExamResponse

```
{
  "exam_id": str,
  "student_id": str,
  "exam_version": int,
  "responses": List[QuestionResponse],
  "overall_feedback": str,
  "strengths": List[str],
  "weaknesses": List[str],
  "recommendations": List[str],
  "submitted_at": datetime
}
```

Purpose: Represents a complete exam session containing all questions, student responses, and grading results for a single attempt.

4.1.2 QuestionResponse

```
{
  "question_id": int,  # Unique identifier
  "question": str,
  "student_answer": str,
  "grading": GradingResult,
  "question_type": str,
  "difficulty": str,
  "time_spent": int,  # Time in seconds
  "attempts": int,
}
```

Purpose: Represents a single exam question with its associated grading criteria and student information.

4.1.3 GradingResult

```
{
  "criteria": List[str],
  "scores": {criterion: int, ...},
  "total_score": int,
  "max_possible": int,
  "feedback": str,
}
```

Purpose: Defines the grading criteria and point allocation for evaluating student responses.

4.1.4 ExamEvaluationResponse

```
{
  "evaluations": List[...],
  "total_processed": int,
  "successful": int,
  "failed": int,
}
```

Purpose: Provides context and background information to fully students understand the question domain.

4.1.5 QuestionResponse

```
{
  "question_id": int,
  "student_answer": str,
  "time_spent": int,
  "hint_used": bool,
  "additional_notes": str
}
```

Purpose: Stores a student's complete response to a specific question.

4.1.6 ExamResult

```
{
  "exam_id": str,
  "student_id": str,
  "overall_score": float,
  "question_results": List[...],
  "completion_time": int,
  "difficulty_adjustment": str,
  "next_recommendation": str
}
```

Purpose: Contains the complete grading result for a student response, including points, percentage, and detailed feedback.

4.1.7 CriteriaEvaluation

```
{
  "criteria": str,
  "score": int,
  "max": int,
  "feedback": str,
}
```

Purpose: Represents the grade for a single grading criterion.

4.2 Data Storage

Current Implementation:

- **Storage Type:** In-memory Python dictionary (`evaluations`, `exam_sessions`)
- **Persistence:** Data lost on server restart
- **Performance:** Fast lookup for prototyping
- **Scalability:** Not suitable for production
- **Future Enhancement:** Will be replaced by PostgreSQL in production

5. System Workflows

5.1 Question Generation Workflow

1. **Trigger:** Teacher initiates exam creation
2. **Parameters:** Subject, difficulty, question count, question types
3. **Prompt Construction:** Template selection based on question type
4. **Service Initialization:** Backend generates unique session ID, AI-GRE Criteria Evaluation object , Initializes empty lists for questions and responses
5. **LLM Generation:** Calls OpenAI API with structured prompt
6. **Response Parsing:** Extracts questions from LLM response
7. **Response Parsing:** Extracts questions from LLM response
8. **Storage:** Saves exam data to in-memory dictionary
9. **Response:** Returns exam ID and question count to frontend

5.2 Grading Workflow

1. **Trigger:** Student submits exam responses
2. **Data Retrieval:** Retrieves exam and question data from storage
3. **Rubric Preparation:** Generates rubrics with intelligent difficulty-aware weighting
4. **Grading Loop:** Processes each response individually
5. **Prompt Construction:** Creates grading prompt with question context, student answer, and rubrics
6. **LLM Evaluation:** Calls OpenAI API for grading
7. **Response Parsing:** Extracts scores and feedback from LLM response
8. **Storage:** Saves grading results to evaluation storage
9. **Response:** Returns grading summary to frontend

5.3 Analytics Workflow

1. **Trigger:** Teacher requests analytics
2. **Data Aggregation:** Collects all evaluations
3. **Statistical Analysis:** Calculates distributions, averages, trends
4. **Report Generation:** Creates comprehensive analytics report
5. **Response:** Returns analytics data to frontend

6. API Design

6.1 REST Endpoints

Base URL: `/api/v1`

Exam Management:

- `POST /exams` Create exam (async mode)
- `GET /exams/{id}` Get exam status (poll for completion)
- `GET /exams/{id}/questions` Get exam questions (when ready)
- `GET /exams` List exams (with filters)

Response Codes:

- `200` Success
- `201` Created
- `202` Accepted (async processing)
- `400` Bad request
- `404` Not found
- `500` Server error
- `503` Service unavailable (LLM error)

6.2 Error Handling

HTTP Status Codes: All errors return clear error messages with details for debugging.

Error Response Format:

```
{
  "detail": "Error message here"
}
```

Common Errors:

- `503 Service unavailable`: LLM API error (check API key, quota, rate limits)
- `404 Not found`: Exam or session ID invalid
- `400 Bad request`: Invalid parameters
- `500 Internal server error`: Unexpected error (check logs)

6.3 Rate Limiting

Current Implementation: None (development mode)

Production Recommendations:

- `/exams`: 10 requests/minute per IP
- `/exams/{id}/evaluate`: 5 requests/minute per IP
- `/exams/{id}/questions`: 60 requests/minute per IP

6.4 Authentication & Security

Current Implementation: None (development mode)

Production Recommendations:

- No authentication required
- Rate limiting prevents DoS attacks
- Input validation via Pydantic models
- No sensitive data in responses
- All errors logged for monitoring

Security Considerations:

- API keys stored in environment variables
- Rate limiting prevents cost amplification attacks
- Input validation prevents injection attacks
- Error messages don't expose internal details

Future Enhancements:

- JWT authentication
- Role-based access control (RBAC)
- API key management

- Rate limiting
- Input sanitization
- SQL injection prevention (when database added)

7. LLM Integration

7.1 Prompt Engineering

7.1.1 Question Generation Prompt Template

The prompt template includes:

- Domain context and scope
- Professor instructions (if provided)
- Target difficulty level
- Explicit formatting requirements (Python dictionary only)
- Example structure for expected response

Key Requirements:

- Response must be ONLY a Python dictionary
- No markdown formatting
- No explanatory text outside dictionary
- Start with `{` and end with `}`

7.1.2 Grading Prompt Template

The grading prompt includes:

- Question text and domain
- Complete grading rubric (criteria, points, required elements)
- Background information provided to student
- Student's response text
- Time spent on question
- Instructions for thorough, fair, constructive evaluation

Key Requirements:

- Consider depth of understanding, not keyword matching
- Provide criterion-by-criterion breakdown
- Identify strengths, weaknesses, and suggestions
- Return Python dictionary with specified schema

7.2 LLM Response Parsing

Multi-Strategy Parsing Approach:

1. **Python `ast.literal_eval()`** - For Python-style dictionaries with single quotes 2. **JSON parsing** - For standard JSON format with double quotes 3. **Regex extraction** - To find dictionary patterns within text 4. **Quote fixing** - Convert single to double quotes and retry 5. **Brace matching** - Extract complete dictionaries from mixed content

Error Handling:

- If all parsing strategies fail, return error dictionary
- Provide user-friendly error messages
- Don't expose raw LLM responses in production systems
- Allow exam to continue without crashing

7.3 API Configuration

Provider: Together.ai **Endpoint:** `https://api.together.xyz/v1/chat/completions` **Model:** `mistralai/Mixtral-8x7B-Instruct-v0.1` **Authentication:** Bearer token in `Authorization` header **Request Format:** JSON with system/user messages

Configuration:

- Question Generation: Temperature 0.8, Max Tokens 3000
- Response Grading: Temperature 0.3, Max Tokens 3000

8. Class Structure & OOP Design

8.1 Class Hierarchy

```
Base Models (Pydantic):
│
├── ExamSession
├── ExamQuestion
├── GradingRubric
├── DomainInformation
├── StudentResponse
├── GradeResult
├── GradeExplanation
└── CriterionGrade


Business Logic Classes:
├── LLMClient
│   ├── generate_question()
│   ├── grade_response()
│   └── _extract_python_dict()
│
├── PromptBuilder
│   ├── format_question_generation_prompt()
│   └── format_grading_prompt()
│
└── SessionManager
    ├── create_session()
    ├── get_session()
    └── update_session()
```

8.2 Key Classes

8.2.1 LLMClient

Responsibility: Manage LLM API interactions **Methods:**

- `generate_question(domain, professor_instructions, difficulty) -> dict`
- `grade_response(question, rubric, student_response) -> dict`
- `_extract_python_dict(response_text) -> dict` (private)

Design Principles:

- Single Responsibility: Only handles LLM communication
- Error Handling: Graceful degradation with user-friendly messages

8.2.2 PromptBuilder

Responsibility: Construct LLM prompts from templates **Methods:**

- `format_question_generation_prompt(domain, instructions, difficulty) -> str`
- `format_grading_prompt(question, rubric, response) -> str`

Design Principles:

- Separation of Concerns: Prompt logic isolated from API calls
- Template-based: Easy to modify prompts without changing code

8.2.3 SessionManager

Responsibility: Manage exam sessions in memory **Methods:**

- `create_session(student_id, domain, num_questions) -> ExamSession`
- `get_session(session_id) -> Optional[ExamSession]`
- `update_session(session_id, response, grade) -> None`

Design Principles:

- Encapsulation: Session storage details hidden from consumers
- Simple interface for session operations

8.3 OOP Principles Applied

1. **Single Responsibility Principle (SRP)** - Each class has one clear purpose - LLMClient handles API calls, PromptBuilder handles prompts, SessionManager handles sessions
2. **Separation of Concerns** - Data models separate from business logic - Frontend templates separate from backend API
3. **Data Encapsulation** - Pydantic models enforce data validation - Private methods (`_extract_python_dict`) hide implementation details
4. **Error Handling** - Consistent error response format - User-friendly messages without exposing internals

9. Dependencies

9.1 Python Packages

```
fastapi==0.104.0          # Web framework

uvicorn[standard]==0.24.0 # ASGI server
pydantic==2.5.0           # Data validation
pydantic-settings==2.0.3  # Configuration management
jinja2==3.1.2             # Template engine
requests==2.31.0          # HTTP library for LLM API
python-multipart==0.0.6   # Form data parsing
```

9.2 External Services

- **Together.ai API:** Requires API key (Bearer token)
- **Python 3.9+:** Runtime environment

10. Configuration

10.1 Environment Variables

```
TOGETHER_API_KEY=     # Required for LLM service

HOST=127.0.0.1                # Server host (default: 127.0.0.1)
PORT=8000                     # Server port (default: 8000)
DEBUG=false                   # Debug mode (default: false)
```

10.2 Configuration Management

- Uses Pydantic Settings for environment variable loading
- Supports `.env` file for local development
- Configuration validated at application startup

11. Testing Strategy

11.1 Unit Testing

- Test individual classes and methods in isolation
- Mock LLM API calls for predictable testing
- Test prompt formatting with various inputs
- Test response parsing with different formats

11.2 Integration Testing

- Test complete workflows (question generation → response → grading)
- Test API endpoints with mock data
- Test error handling scenarios

11.3 Manual Testing

- Test UI flows through browser
- Verify LLM responses are correctly parsed
- Test edge cases (empty responses, parsing failures)

12. Deployment

12.1 Development

Local Development:

```
uvicorn main:app --reload --host 127.0.0.1 --port 8000
```

12.2 Production Considerations

Recommended Platforms:

- Heroku
- AWS (Elastic Beanstalk, EC2)
- DigitalOcean
- Railway
- Render

Deployment Requirements:

- Python 3.9+ runtime
- Environment variables for API keys
- Persistent storage if database added (currently in-memory only)

Scaling Considerations:

- Current in-memory storage doesn't scale horizontally
- Database required for multi-instance deployment
- LLM API rate limits may require queuing system

13. Future Enhancements

13.1 Planned Features (From Requirements)

1. **Follow-up Questions** - LLM generates follow-up questions for unsatisfactory responses - Multiple rounds of questions before final grading
2. **Final Grade Compilation** - LLM evaluates all question grades to determine final exam grade - Multiple grading schemes selectable by professor
3. **Curved Grading** - Compare responses across all students - Partial-order sorting for relative grading - Include historical exam data
4. **Dispute System** - Allow students to dispute grading - AI assessment of dispute validity - Professor review interface
5. **Discussion-Based Testing** - Conversational exam interface - History tracking for multi-turn dialogues - Tests understanding through discussion
6. **Database Persistence** - SQLite for session storage - Exam history and analytics - Professor review capabilities

13.2 Technical Improvements

- Authentication and authorization
- Rate limiting
- Input sanitization
- Caching for frequently used prompts
- Monitoring and logging
- API response validation enhancements

14. Risk Assessment

14.1 Technical Risks

1. **LLM API Availability** - **Risk:** Service downtime affects all operations - **Mitigation:** Error handling, user-friendly messages
2. **Response Parsing Failures** - **Risk:** LLM doesn't return expected format - **Mitigation:** Multiple parsing strategies, graceful degradation
3. **Data Loss** - **Risk:** In-memory storage lost on restart - **Mitigation:** Documented limitation, database planned

14.2 Operational Risks

1. **API Rate Limits** - **Risk:** Too many requests to LLM service - **Mitigation:** Monitor usage, implement queuing if needed
2. **Security Vulnerabilities** - **Risk:** No authentication, session hijacking possible - **Mitigation:** Documented, planned for future enhancement

15. Requirements Coverage

15.1 Core Requirements

All core requirements from the original specification are covered:

- ✅ **Question Generation** Real-time LLM-based question generation with rubrics
- ✅ **Response Grading** Automated grading with detailed feedback
- ✅ **Session Management** In-memory session storage and retrieval
- ✅ **Data Models** Complete Pydantic models for all data structures
- ✅ **API Endpoints** All required REST endpoints implemented
- ✅ **Frontend Pages** All four required HTML pages (home, create, exam, results)
- ✅ **Error Handling** Comprehensive error handling with user-friendly messages
- ✅ **LLM Integration** Full integration with Together.ai API

15.2 Advanced Features (Deferred)

The following advanced features are documented but deferred to future releases:

- Follow-up questions for insufficient responses
- Final grade compilation via LLM
- Curved grading across all students
- Dispute resolution system
- Discussion-based testing
- Database persistence

Rationale: These features are out of scope for the MVP but are designed for future enhancement.

16. OOP Utilization Summary

16.1 Object-Oriented Principles

The design follows strong OOP principles:

Encapsulation:

- Data and methods bundled in classes
- Private methods hide implementation details
- Public interfaces provide controlled access

Inheritance:

- All data models inherit from Pydantic BaseModel
- Inherit automatic validation, serialization, type checking

Polymorphism:

- All models share common BaseModel interface
- Same methods work across different model types

Abstraction:

- Complex LLM API interaction abstracted into simple method calls
- Storage details abstracted away from consumers

16.2 SOLID Principles

Single Responsibility Principle (SRP): ✅ Excellent

- Each class has one clear purpose
- LLMClient handles API calls, PromptBuilder handles prompts, SessionManager handles sessions

Open/Closed Principle (OCP): ✅ Good

- Extensible without modification
- Can add new providers/strategies without changing core code

Liskov Substitution Principle (LSP): ✅ Excellent

- All Pydantic model subclasses are substitutable

Interface Segregation Principle (ISP): ✅ Good

- Focused interfaces, no god objects

Dependency Inversion Principle (DIP): ✅ Good

- Dependencies point to abstractions, not concrete implementations

17. Appendices

17.1 Glossary

- **LLM:** Large Language Model
- **SRP:** Single Responsibility Principle
- **UUID:** Universally Unique Identifier
- **API:** Application Programming Interface
- **REST:** Representational State Transfer
- **ASGI:** Asynchronous Server Gateway Interface

17.2 References

- FastAPI Documentation: https://fastapi.tiangolo.com/
- Together.ai API Documentation: https://docs.together.ai/docs
- Pydantic Documentation: https://docs.pydantic.dev/
- Mermaid Diagram Syntax: https://mermaid.js.org/

17.3 UML Diagram Locations

All UML diagrams referenced in this document are available as PNG images in the `uml_exports/` directory:

- Class Diagrams: `11_complete_class_diagram_unknown.png`
- Component Diagrams: `21_System_Architecture_Components_activity.png`
- Sequence Diagrams: `31_Question_Generation_Sequence_activity.png`
- Activity Diagrams: `41_Question_Generation_Activity_Flow_activity.png`
- Use Case Diagrams: `51_System_Use_Cases_unknown.png`
- Deployment Diagrams: `61_System_Deployment_unknown.png`

For complete UML documentation, see `Programming_Design_Document_UML.md`.

17.4 Document History

- **Version 1.0:** Initial design document
- **Version 2.0:** Comprehensive edition with UML diagrams and additional technical details